B.TECH 3RD-YEAR ACADEMIC PROJECT BLUEPRINT

AI-Based Indian Sign Language Alphabet Recognition and Voice Conversion System

Comprehensive Project Architecture, Folder Structure, Implementation Roadmap & Viva Strategy

Target Scope: ISL Fingerspelling A–Z (Educational Prototype)

Department: B.Tech Computer Science / AI & ML

Core ML Model: TensorFlow/Keras Convolutional Neural Network (CNN)

Web Stack: React + Vite + Tailwind (Frontend) | FastAPI (Backend)

Voice Conversion: Browser Native Web Speech API (SpeechSynthesis)

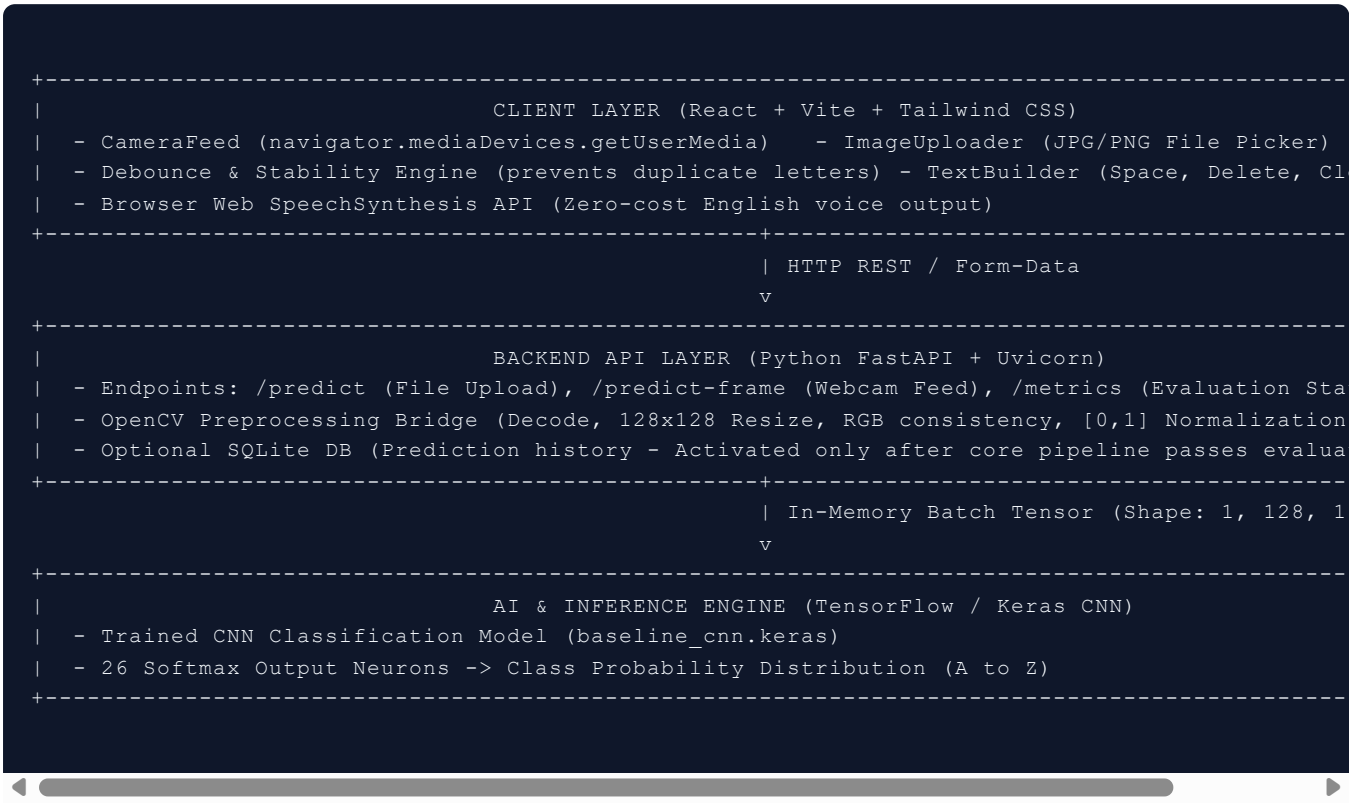
Planned Primary Dataset: RealSign ISL (A–Z) + Consent-based Volunteer Data

Document Purpose: Yeh document is project ka master technical reference blueprint hai. Iske andar complete software architecture, image-to-voice dataflow, folder structure, 9-phase execution roadmap, dataset audit gate, aur team viva preparation guide comprehensively documented hain.

Academic Scope Limitation Note: Yeh system alphabet-level fingerspelling gestures (A–Z) ke liye educational prototype hai. Yeh full continuous natural Indian Sign Language sentences translate nahi karta (jo ki facial expressions, body posture, two-handed motion grammar aur temporal context par depend karte hain).

1. System Architecture & High-Level Flow

Project ek clean **Three-Tier Decoupled Architecture** follow karta hai taaki frontend presentation, backend REST handling aur deep learning inference modules independent rahein:



End-to-End Gesture-to-Voice Workflow

1. **Input Acquisition:** User camera ke samne ISL hand sign perform karta hai ya local image upload karta hai. Browser continuous video frames capture karta hai.
2. **OpenCV Preprocessing:** Backend frame receive karke image decode karta hai, uniform channels ensure karta hai, proposed standard size (initial proposal: 128 × 128) par resize karta hai, aur pixel values ko [0, 1] me normalize karta hai.
3. **CNN Inference:** Model processed tensor par forward pass chalakar 26 classes ke confidence scores return karta hai.
4. **Debounce & Stability Logic (Frontend):** Real-time streaming me multiple identical frames aate hain. Stability window ensure karti hai ki candidate letter tabhi append ho jab woh consecutive frames tak consistent ho, aur hand change hone tak repeat na ho.
5. **Voice Conversion:** User jab "Speak" press karta hai, browser ki built-in `window.speechSynthesis` API accumulated text ko bina kisi paid external cloud API ke bolkar sunati hai.

2. Parameter Status & Dataset Audit Gate

Proposed vs Confirmed Parameters Policy:

- **Image Resolution:** 128 × 128 initial proposed baseline hai. Dataset inspection aur model experiment ke baad final resolution confirm hogi.
- **Confidence Threshold & Frame Interval:** 75–80% confidence aur 300–500ms sampling rate proposed values hain. Live webcam testing me response dekhkar tune kiya jayega.
- **Inference Latency:** Koi unverified number (jaise 20–40ms) assume nahi kiya gaya hai. System deploy hone ke baad actual hardware par latency measure karke report ki jayegi.
- **Sample JSON Confidences:** API documentation me sample outputs sirf illustrative schema clarification ke liye hain.

Critical Dataset Audit Gate: Movement-Dependent Signs (J & Z)

Indian Sign Language me kuch letters inherently dynamic motion (movement-based) hote hain (jaise J aur Z):

- **Static CNN Limitation:** Ek 2D static image CNN time-dependent motion ko directly capture nahi kar sakta.
- **Audit Mandate:** Training start karne se pehle dataset me J aur Z ke image samples ko audit kiya jayega ki dataset unhe single static pose me represent karta hai ya stroke keyframe me.
- **Honest Academic Reporting:** Documentation aur viva presentation me clearly acknowledge kiya jayega ki single still-image model dynamic movement capture nahi karta, aur ise future scope ke roop me define karenge.

3. Complete Proposed Folder Structure

Codebase ko clean, modular aur B.Tech academic evaluation ke liye well-organized rakha gaya hai:

```
Sign-Language-Alphabet-Recognition/
├── .gitignore                      # Excludes dataset/, .venv/, caches, node_modules, heavy m
├── README.md                       # Project overview, installation steps, and live implement
│
├── dataset/                        # LOCAL ONLY (Never committed to Git repository)
│   ├── raw/                        # Downloaded RealSign public dataset (train/val/test split
│   ├── custom/                     # Volunteer-collected images with proper consent documenta
│   └── dataset_info.md             # Dataset audit report: image counts, dimensions, licence
│
├── training/                       # ML Pipeline scripts (Transparent & student-explainable)
│   ├── __init__.py
│   ├── inspect_dataset.py          # Audit script: verifies labels, corrupt files & J/Z repre
│   ├── preprocess.py               # Image resizing, normalization, and safe data augmentatio
│   ├── model.py                    # Proposed Keras CNN architecture definition
│   ├── train.py                    # Training execution, validation monitoring, and saving we
│   └── evaluate.py                 # Evaluates test split, generates real confusion matrix &
│
├── saved_models/                   # Exported trained artifacts
│   ├── baseline_cnn.keras          # Main trained model weights (saved after genuine training
│   └── label_classes.json          # Index to letter mapping (0 -> 'A', ..., 25 -> 'Z')
│
├── results/                        # Generated evaluation artifacts (100% Genuine, No Fake Da
│   ├── confusion_matrix.png        # Generated plot from test set evaluation
│   ├── accuracy_loss_curves.png    # Training vs validation performance graphs
│   └── evaluation_metrics.json     # Real test set accuracy, precision, recall, and F1-scores
│
├── backend/                        # FastAPI Application
│   ├── requirements.txt            # Backend dependencies (fastapi, uvicorn, opencv-python, t
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py                 # FastAPI initialization and CORS setup
│   │   ├── config.py               # Proposed thresholds, paths, and class configurations
│   │   ├── routes/
│   │   │   ├── predict.py          # /predict (Image Upload) and /predict-frame (Webcam Feed)
│   │   │   ├── metrics.py          # /metrics (Serves saved evaluation results to frontend)
│   │   │   └── history.py          # /history (Optional SQLite prediction logging)
│   │   ├── services/
│   │   │   ├── model_service.py    # Singleton model loader and inference runner
│   │   │   └── preprocessor.py     # OpenCV decoding and frame normalization
│   │   └── db/                     # Optional database module (enabled only after core pipeli
│   │       ├── database.py         # SQLite connection setup
│   │       └── models.py           # Prediction history schema
│   └── tests/
│       ├── test_health.py          # Health-check route verification
│       └── test_predict.py         # Inference endpoint test with sample image
│
├── frontend/                       # React + Vite + Tailwind CSS Application
│   ├── package.json
│   ├── vite.config.js
│   ├── tailwind.config.js
│   └── src/
│       ├── main.jsx / App.jsx      # Application bootstrap and routing
│       ├── components/
│       │   ├── Navbar.jsx          # Header with live system status badge
│       │   ├── CameraFeed.jsx      # Webcam stream capture component
│       │   ├── ImageUploader.jsx   # Drag & drop / file picker for image testing
│       │   ├── TextBuilder.jsx     # Accumulated word display with Space/Delete/Clear buttons
│       │   └── VoiceController.jsx # Speak and Stop speech synthesis triggers
│       ├── pages/
│       │   ├── Home.jsx            # Project overview and interactive workflow guide
│       │   ├── Recognition.jsx     # Core recognition workspace (Live Webcam & Upload)
```

```
│       │   ├── Dashboard.jsx       # Real evaluation metrics and session counters
│       │   ├── Learning.jsx        # ISL A-Z alphabet reference guide & flashcards
│       │   ├── Performance.jsx     # Confusion matrix and precision/recall analysis
│       │   └── About.jsx           # Problem statement, team, architecture & limitations
│       ├── services/               # api.js (Axios client) & speechService.js (Web Speech API
│       └── utils/debounce.js       # Real-time gesture stabilization and repeat-lock logic
│
└── docs/                           # Documentation & Academic Preparation
    ├── architecture_diagram.png    # System architecture graphic
    ├── api_specification.md        # REST API request/response documentation
    └── team-knowledge-hub/         # Dedicated Team Knowledge & Viva Preparation Center
        ├── START_HERE_TEAM_GUIDE.md # Master team guide: quickstart, responsibilities, runnin
        ├── setup_guide.md          # Step-by-step local development setup (Python, Node.js, G
        ├── workflow.md             # Git discipline, commit standards, branch management
        ├── learning_roadmap.md     # CNN principles, OpenCV fundamentals, and ISL concepts
        ├── code_explanations.md    # Code walkthrough of ML, API, and UI modules for viva
        └── viva_preparation.md     # Comprehensive viva Q&A, examiner FAQs, limitation defens
```

4. 9-Phase Implementation Roadmap & Status Tracking

Har phase me systematic verification tasks define kiye gaye hain. Kisi bhi phase ko untested complete declare nahi kiya jayega:

Phase	Work & Key Tasks	Planned Deliverable	Status
Phase 1: Setup & Foundation	Git repository configuration, comprehensive `.gitignore`, project directories, Python virtual environment, frontend setup (Vite + React + Tailwind), and initial `docs/team-knowledge-hub/`.	Working dev environment & clean initial commit.	PLANNED
Phase 2: Dataset Audit Gate	Download RealSign dataset, run `inspect_dataset.py` to check image counts and dimensions, audit dynamic classes (J & Z), formulate volunteer collection consent protocol.	`dataset_info.md` audit report with dynamic signs notes.	PLANNED
Phase 3: Image Preprocessing	OpenCV preprocessing module: 128×128 resizing, normalization to [0, 1], RGB channel verification, safe augmentation (rotation ±10°, NO horizontal flipping).	Verified `preprocess.py` with train/val/test data loaders.	PLANNED
Phase 4: CNN Training	Proposed CNN architecture in Keras (Conv2D blocks, Dropout, Dense + Softmax 26 classes), EarlyStopping callbacks, training execution, model export.	Trained `baseline_cnn.keras` with training logs.	PLANNED
Phase 5: Evaluation & Metrics	Evaluate on unseen test data, compute Accuracy, Precision, Recall, F1-scores, generate confusion matrix and loss/accuracy curves. **Zero fake numbers.**	Genuine `results/` evaluation artifacts.	PLANNED
Phase 6: FastAPI Backend	FastAPI routes (`/health`, `/classes`, `/predict`, `/predict-frame`, `/metrics`), model singleton service, error handling. SQLite kept optional.	Tested API with interactive Swagger docs (`/docs`).	PLANNED
Phase 7: React Frontend	Modern responsive UI: Navbar, Home, Learning (A-Z cards), Recognition workspace, Dashboard, Performance, and About screens with clear placeholders.	Responsive web interface ready for integration.	PLANNED
Phase 8: Integration & Voice	Connect webcam feed to backend inference, integrate debounce stability logic in TextBuilder, connect browser Web SpeechSynthesis API for voice output.	End-to-end working prototype (Gesture → Text → Voice).	PLANNED
Phase 9: Docs & Viva Prep	README.md, finalize `team-knowledge-hub/`, 15–18 slide presentation outline, unseen-person evaluation analysis, viva defense questions.	Academic submission package ready for demonstration.	PLANNED

5. Team Responsibilities & Work Breakdown

Academic group evaluation ke liye team members ke beech roles ka clear division kiya gaya hai:

Role & Ownership	Core Responsibilities	Viva Focus Area
Member 1: Machine Learning & Preprocessing	Dataset Audit Gate, Preprocessing pipeline, CNN model architecture, training loop, hyperparameter tuning, confusion matrix generation.	CNN layers, convolution mathematics, activation functions, overfitting control, data augmentation reasons.
Member 2: Backend & System Architecture	FastAPI endpoints, OpenCV image decoding, model singleton inference service, CORS, error handling, optional SQLite logging.	REST API design, asynchronous processing, model memory management, backend preprocessing pipeline.
Member 3: Frontend & Human Interaction	React component hierarchy, webcam stream integration, debounce & stability logic, TextBuilder controls, Web Speech API.	Browser camera APIs, debounce algorithm, state management, client-side Text-to-Speech synthesis.
Member 4: Quality, Testing & Viva Hub	Volunteer dataset collection & consent, unseen signer testing, `team-knowledge-hub` documentation, slide deck, project report.	System limitations, static vs dynamic ISL signs, dataset bias, evaluation metrics interpretation.

6. Git Discipline & Academic Integrity Rules

- **Strict Git Exclusions:** Large image datasets (`dataset/`), virtual environments (`.venv/`), Python byte caches (`__pycache__/`), `node_modules/` aur heavy intermediate model checkpoints Git me commit nahi honge.
- **Commit Conventions:** Standard prefixes follow honge: `chore:` (setup/config), `data:` (audit/inspection), `feat:` (features), `fix:` (bug fixes), `docs:` (documentation).
- **Remote Repository Safety:** User ke explicit permission ke bina remote repo par push nahi kiya jayega.
- **No Fabricated Results:** Dashboard aur presentation ke evaluation metrics 100% actual test runs se aayenge. Incomplete features ko clearly "*Planned / Under Development*" mark kiya jayega.